7. HTML — Multimedia & Modern HTML

Index: HTML tutorial
Previous: Semantic HTML
Next: Accessibility and SEO Basics

The clip in this module is a short HR policy recording. The dialog says Confirm leave request.

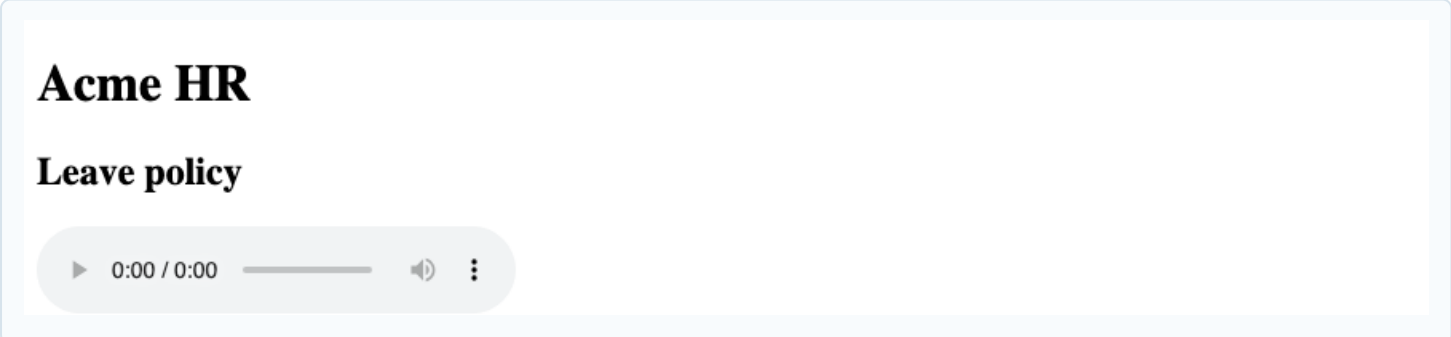
1. `<audio>`

`<audio>` plays a sound file. `controls` shows the browser's play bar. The HR policy clip can be audio when a video is not needed. Text of the policy still belongs on the page. The recording is an extra, not the only copy.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Policy audio</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <h2>Leave policy</h2>
    <audio controls src="media/leave-policy.mp3">
      A short HR policy clip.
    </audio>
  </main>
</body>
</html>
```

This will produce the following result:



2. `<video>`

`<video>` plays a picture with sound. `controls` shows the play bar. `width` and `height` reserve the box. The same leave policy can be a short clip. The words inside the element are a fallback only when the browser does not support `<video>` at all. A missing or broken file does not show that text; the player stays empty or shows its own error.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Policy clip</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <h2>Leave policy</h2>
    <video controls width="320" height="180" src="media/leave-policy.mp4">
      A short HR policy clip.
    </video>
  </main>
</body>
</html>
```

This will produce the following result:



3. `<source>`

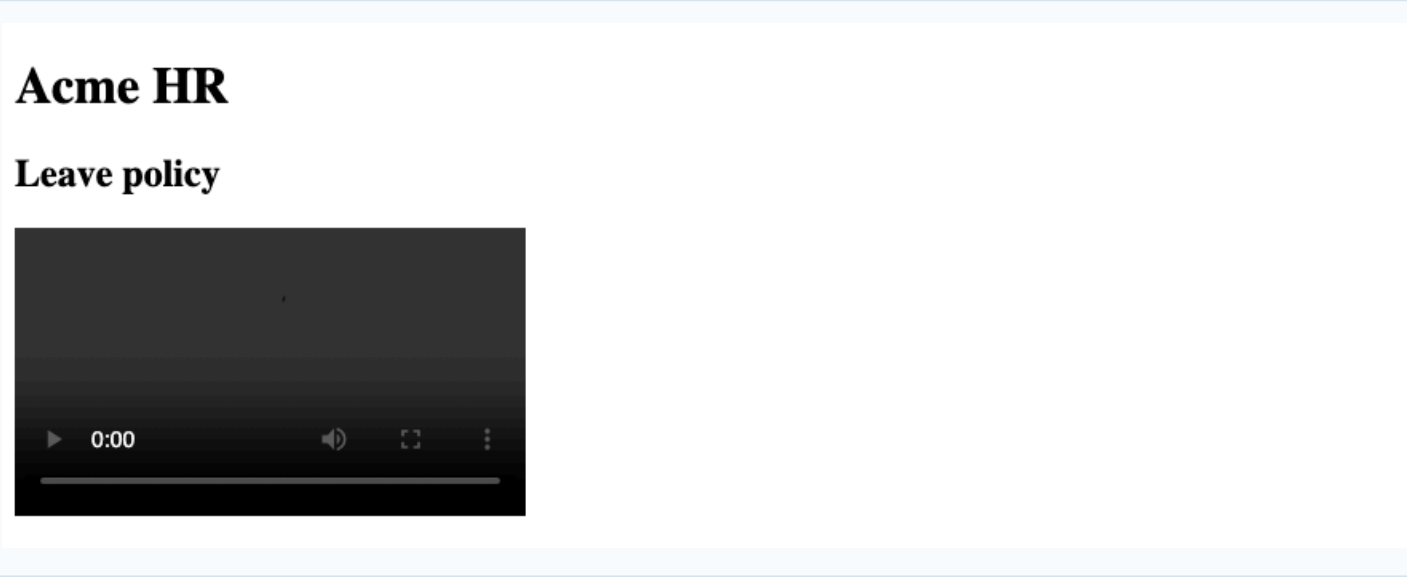
`<source>` lists a file inside `<audio>` or `<video>`. The browser picks the first type it can play. `type` is the media type, such as `video/mp4`. More than one source covers more browsers. The fallback text still sits inside the media element.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Sources</title>
</head>
<body>
  <header>
```

```
    <h1>Acme HR</h1>
  </header>
  <main>
    <h2>Leave policy</h2>
    <video controls width="320" height="180">
      <source src="media/leave-policy.webm" type="video/webm">
      <source src="media/leave-policy.mp4" type="video/mp4">
      A short HR policy clip.
    </video>
  </main>
</body>
</html>
```

This will produce the following result:



4. `<track>`

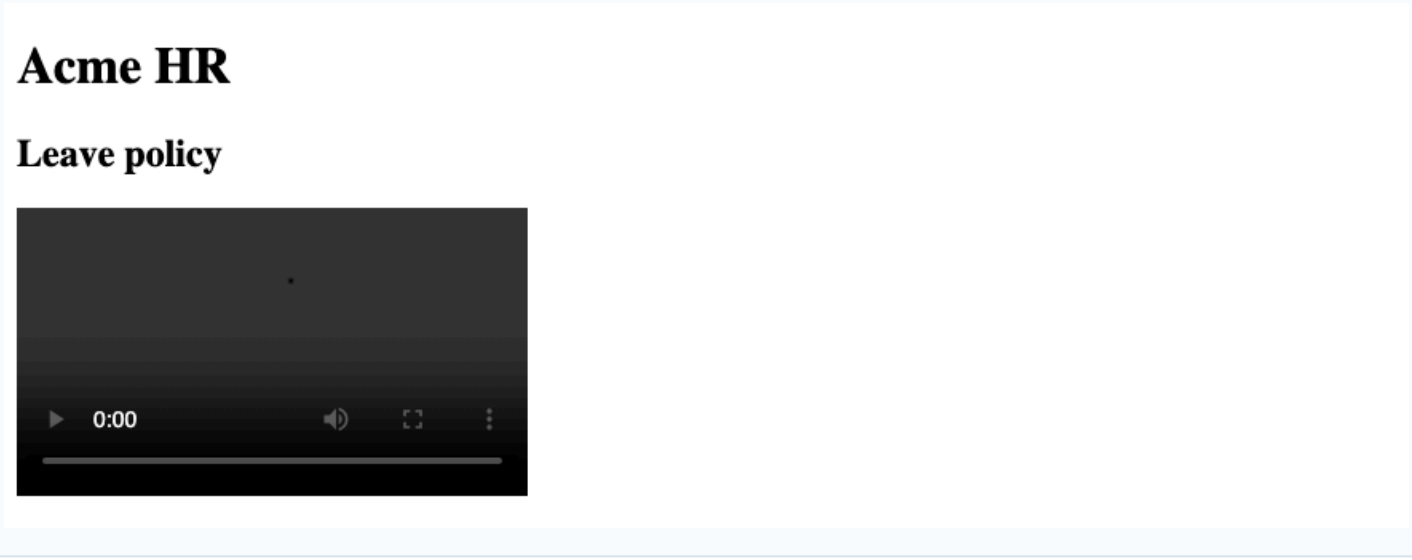
`<track>` attaches a timed text file, such as captions. `kind="captions"` is the dialogue and the sounds. `srclang` is the language. `default` marks the track to show first. The policy clip has an English caption file. Captions are how the clip is read without sound.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Captions</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <h2>Leave policy</h2>
    <video controls width="320" height="180">
      <source src="media/leave-policy.mp4" type="video/mp4">
      <track src="media/leave-policy.vtt" kind="captions" srclang="en" label="English" default>
      A short HR policy clip.
    </video>
  </main>
```

```
</body>
</html>
```

This will produce the following result:



5. Audio & Video Attributes

`controls` shows the player. `poster` is an image shown before the video plays. `preload` hints how much to fetch: `none`, `metadata`, or `auto`. `loop` and `autoplay` repeat or start on their own. Do not autoplay the policy clip. The employee starts it with the controls.

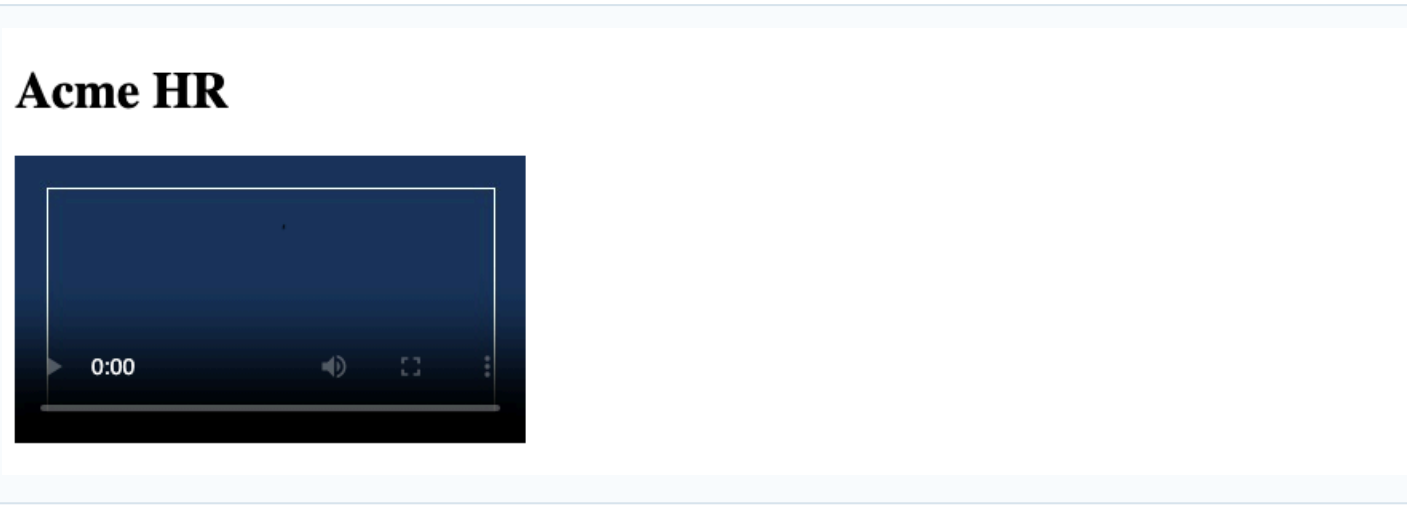
Attribute	Use on the policy clip
`controls`	Show the play bar
`poster`	A still frame before play
`preload="metadata"`	Load duration, not the whole file
`width`, `height`	Reserve the box

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Player attributes</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <video controls width="320" height="180" poster="images/policy-poster.jpg" preload="metadata">
      <source src="media/leave-policy.mp4" type="video/mp4">
      A short HR policy clip.
    </video>
  </main>
```

```
</body>
</html>
```

This will produce the following result:



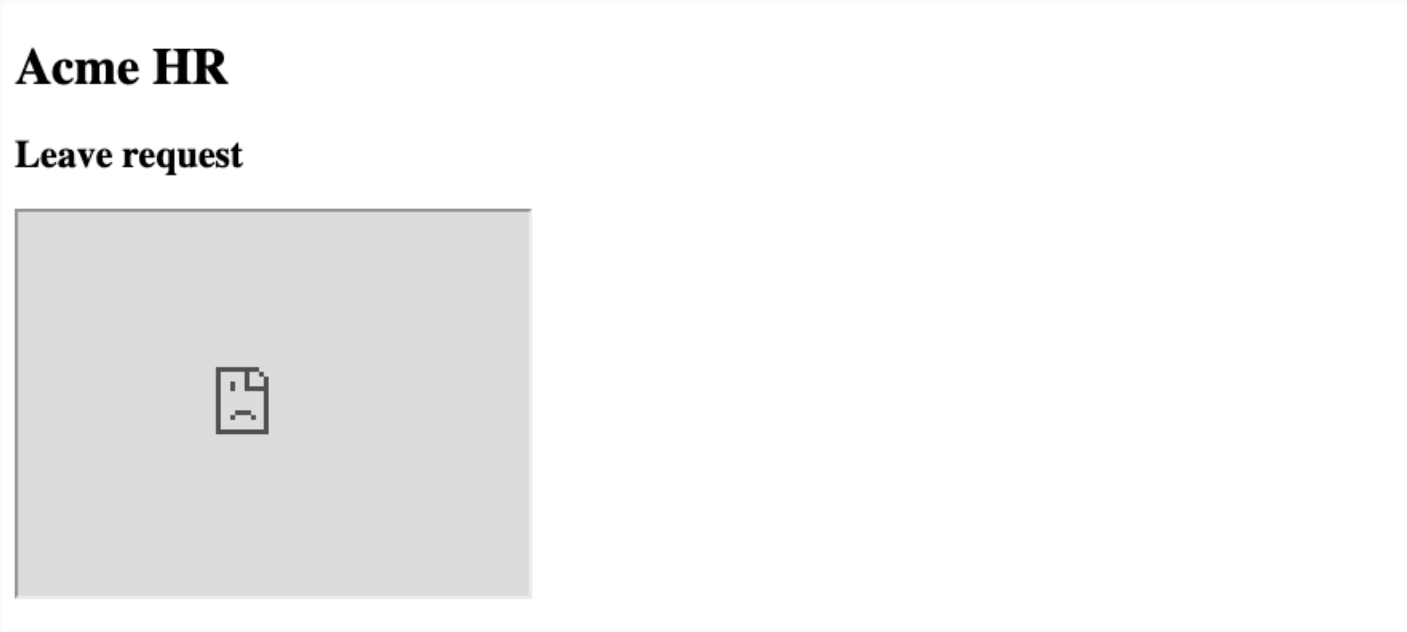
6. `<iframe>`

`<iframe>` embeds another HTML page in a rectangle. `title` names the frame for assistive technology. `width` and `height` set the box. A holiday list page can sit in a frame on the leave page. The title is "Hyderabad holiday list".

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Holiday list</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <h2>Leave request</h2>
    <iframe src="holiday-list.html" title="Hyderabad holiday list" width="320" height="240"></iframe>
  </main>
</body>
</html>
```

This will produce the following result:



7. Embedding External Content

An external page uses an absolute URL in `src`. Trust the page you embed. It runs in your layout. `referrerpolicy` and `sandbox` limit what that page can do. A holiday list on example.com is enough. Do not embed a page you would not open yourself.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — External frame</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <iframe src="https://example.com/holiday-list" title="Holiday list" width="320" height="240" sandbox
  </main>
</body>
</html>
```

This will produce the following result:



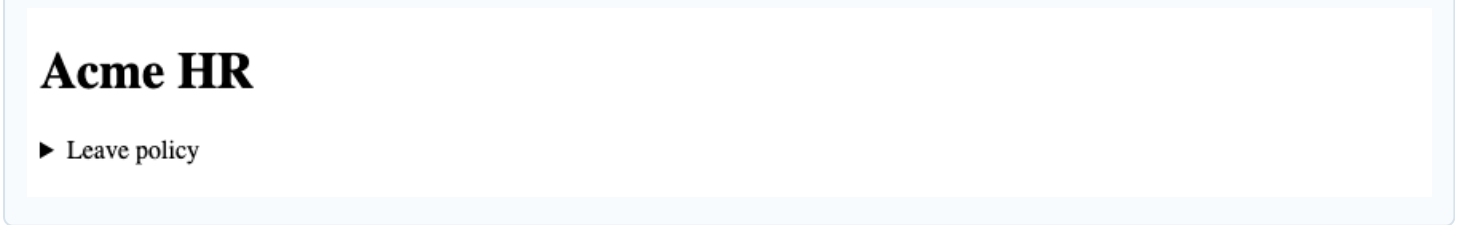
8. `<details>`

`<details>` is a disclosure box. The content is hidden until the reader opens it. The leave policy text can sit in one, under the clip. The employee who already knows the rule can leave it closed. The summary in the next section is the visible label.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Details</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <details>
      <summary>Leave policy</summary>
      <p>Casual leave is filed two days before the start date.</p>
    </details>
  </main>
</body>
</html>
```

This will produce the following result:



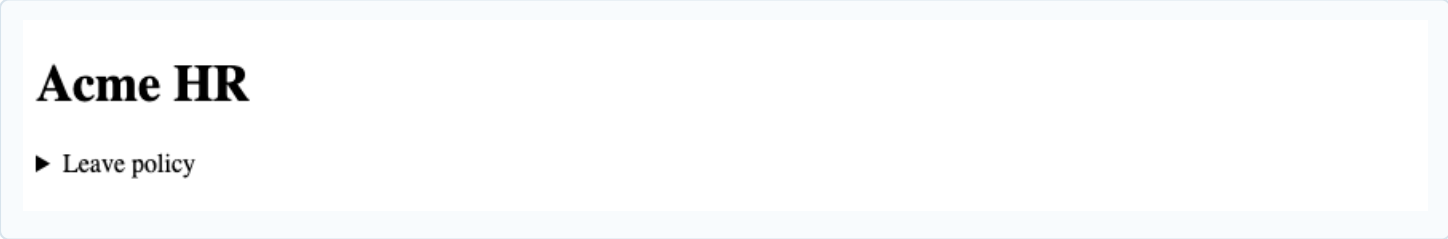
9. `<summary>`

`<summary>` is the label of a `<details>` element. It is the first child. The rest of the details is the hidden part. Write the summary as the question or the name of the note. "Leave policy" is the label. The rule itself is the paragraph inside.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Summary</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <details>
      <summary>Leave policy</summary>
      <p>A medical certificate is required for sick leave over two days.</p>
    </details>
  </main>
</body>
</html>
```

This will produce the following result:



10. `<dialog>`

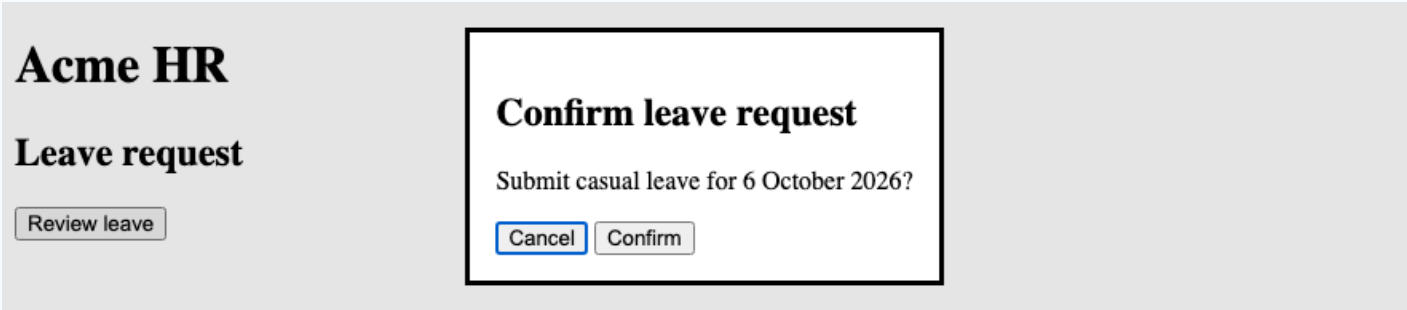
`<dialog>` is a window on the page. The `open` attribute shows it as non-modal: the rest of the page still works. A modal can open without your own script: a button with `command="show-modal"` and `commandfor` pointing at the dialog id invokes it. `showModal()` in script is the older path and still works as a fallback. A lightweight alternative for non-modal overlays is the `popover` attribute. The heading inside the dialog is Confirm leave request. `method="dialog"` on a form closes the dialog and records the button value. Give every button an explicit `type`.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Confirm leave</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
```

```
  </header>
  <main>
    <h2>Leave request</h2>
    <button type="button" command="show-modal" commandfor="confirm-leave">Review leave</button>
    <dialog id="confirm-leave">
      <h2>Confirm leave request</h2>
      <p>Submit casual leave for 6 October 2026?</p>
      <form method="dialog">
        <button type="submit" value="cancel">Cancel</button>
        <button type="submit" value="confirm">Confirm</button>
      </form>
    </dialog>
  </main>
</body>
</html>
```

This will produce the following result:



11. Native HTML Interactivity

`<details>`, form validation, `popover`, and invoker buttons (`command` / `commandfor`) work without a custom script. A modal `<dialog>` can open the same way; `showModal()` remains the script fallback. The `open` attribute is the non-modal dialog path and needs no script. Prefer these elements before adding a script for the same job. The leave form's `required` checks are the same kind of built-in behaviour.

Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme HR — Built-in behaviour</title>
</head>
<body>
  <header>
    <h1>Acme HR</h1>
  </header>
  <main>
    <details>
      <summary>Leave policy</summary>
      <p>A short HR policy clip is on this page.</p>
    </details>
    <dialog open>
      <h2>Confirm leave request</h2>
      <p>This open dialog is non-modal.</p>
    </dialog>
  </main>
```

```
</body>
</html>
```

This will produce the following result:

